



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

TESI DI LAUREA

**VALUTAZIONE DELLA
ROBUSTEZZA DEI TEST GENERATI
DA LLM PER SMART CONTRACT
SOLIDITY TRAMITE MUTATION
TESTING**

RELATORE

Prof. Dario Di Nucci

Dott. Gerardo Iuliano

Università degli Studi di Salerno

CANDIDATO

Francesco Faiella

Matricola: 0512116141

Anno Accademico 2024-2025

Questa tesi è stata realizzata nel

sesa^{lab}
SOFTWARE ENGINEERING
SALERNO

A mia nonna Divina.

Abstract

L'integrità degli Smart Contract su Ethereum rappresenta un requisito critico, poiché la natura immutabile della blockchain trasforma ogni difetto software non rilevato in una vulnerabilità permanente ed economicamente rischiosa. Sebbene l'adozione dei Large Language Models (LLM) prometta di accelerare il ciclo di sviluppo automatizzando la scrittura delle suite di test, la letteratura attuale tende a valutarli principalmente sulla correttezza sintattica, trascurando la loro reale capacità di individuare errori logici profondi.

In questo contesto, la tesi si propone di valutare la robustezza semantica dei test unitari automaticamente generati per Smart Contract in Solidity, mediante l'impiego sistematico di tecniche di Mutation Testing. La metodologia adottata ha previsto lo sviluppo di una pipeline iterativa basata sul modello *open-weights* Qwen 2.5-Coder, progettata per superare i limiti di contesto tipici della generazione automatica. A partire da contratti privi di difetti, sono state generate suite di test validate a *runtime* tramite un meccanismo di *Self-Healing*. Successivamente, sono state create versioni mutate dei contratti originali (tramite il tool SuMo) per verificare la capacità delle test suite di rilevare comportamenti anomali.

I risultati sperimentali dimostrano che, mentre l'approccio iterativo risolve efficacemente le problematiche sintattiche garantendo la compilabilità, la qualità semantica dei test presenta limiti strutturali significativi. Con un Mutation Score medio del 63.5%, le test suite generate si dimostrano efficaci sulla logica matematica, ma tendono a trascurare la verifica degli effetti collaterali (es. emissione di eventi) e delle logiche di trasferimento complesse.

In conclusione, lo studio suggerisce che l'adozione di LLM non debba essere intesa come sostituzione dell'esperto umano, bensì come un cambio di paradigma verso un'ingegneria assistita, in cui l'integrazione di protocolli di validazione indipendenti o l'adozione di modelli di frontiera rimangono requisiti imprescindibili per garantire la sicurezza delle applicazioni blockchain.

Indice

1	Introduzione	1
1.1	Contesto Applicativo	1
1.2	Motivazione e Obiettivo	2
1.3	Risultati Ottenuti	2
1.4	Struttura della Tesi	3
2	Background	4
2.1	Blockchain	4
2.2	Smart Contract su Ethereum	5
2.3	Vulnerabilità negli Smart Contract	6
2.4	Software Testing e Mutation Testing	8
2.5	Large Language Models & Code Generation	10
3	Metodologia Sperimentale	11
3.1	LLM & Dataset	11
3.2	Approccio Zero-shot	12
3.3	Approccio Iterativo	13
3.4	Valutazione dei Empirica dei Test Generati	15
4	Analisi dei Risultati	17
4.1	Risultati sulla generazione dei test case	18

4.1.1	Analisi dei Fallimenti della Fase Zero-Shot	18
4.1.2	Analisi dei Fallimenti Residui nell'Approccio Iterativo	19
4.2	Risultati del mutation testing	20
4.2.1	Analisi dei Mutanti Sopravvissuti	21
5	Discussione	23
6	Conclusioni	25
	Bibliografia	27

CAPITOLO 1

Introduzione

1.1 Contesto Applicativo

Negli ultimi anni, la tecnologia blockchain ha acquisito sempre più notorietà, innescando numerosi progetti in diverse industrie, in particolare nel settore finanziario. Attualmente, il modello tradizionale basato sull'intermediazione fiduciaria è la soluzione dominante per la validazione delle transazioni, ma presenta limiti significativi in termini di costi operativi, latenza e rischi per la controparte in caso di fallimento dell'intermediario [1]. Il successo della blockchain ha favorito l'adozione di tecnologie come gli **Smart Contract** [2]: protocolli informatici che, eseguiti su un'infrastruttura decentralizzata (come la blockchain), automatizzano l'esecuzione di accordi in modo trasparente e immutabile, eliminando la necessità di garanti terzi. Parallelamente, la crescita dei **Large Language Models (LLM)** ha aperto nuove possibilità nell'automazione dello sviluppo software [3], rendendo possibile la scrittura automatica di Smart Contract e, in particolare, delle relative test suite, accelerando così il ciclo di sviluppo e di validazione.

1.2 Motivazione e Obiettivo

L'uso di Large Language Models (LLM) per l'automazione dei task e la generazione di codice rappresenta oggi una frontiera di ricerca attiva e promettente [3]. Tuttavia, l'integrazione di codice generato dall'LLM in un ambiente blockchain comporta rischi elevati. A differenza dello sviluppo software tradizionale, gli Smart Contract sono caratterizzati dall'immutabilità: una volta deployati sulla rete, il codice non può essere modificato né corretto tramite patch successive. Questa irreversibilità implica che qualsiasi vulnerabilità non rilevata in fase di testing si trasformi in una minaccia permanente, con conseguenze potenzialmente disastrose.

Studi preliminari [4] hanno evidenziato che gli attuali LLM tendono a generare codice affetto da gravi problemi di sicurezza, rendendo inaffidabile l'adozione cieca dei loro output. Se gli LLM vengono usati per scrivere anche la test suite del contratto, esiste il rischio concreto di una falsa percezione di sicurezza. L'obiettivo della tesi è valutare i test generati dall'LLM tramite un tool di Mutation Testing. La valutazione dei test non è solo a livello sintattico, ma anche a quello semantico.

1.3 Risultati Ottenuti

I risultati ottenuti dimostrano l'efficacia dell'architettura proposta nel superare le barriere sintattiche: l'implementazione di una pipeline iterativa ha consentito di generare test suite valide e funzionanti, a differenza del fallimento totale dell'approccio base con Zero-Shot. Tuttavia, l'analisi semantica tramite Mutation Testing ha evidenziato limiti strutturali nella qualità del test. Con un Mutation Score del 63.5%, le test suite generate mostrano una buona copertura della logica aritmetica di base, ma tendono a trascurare la verifica degli effetti collaterali (es. eventi) e dei meccanismi di trasferimento, suggerendo che l'LLM operi con un bias verso la "plausibilità" del codice piuttosto che verso la sua "efficacia" di sicurezza.

1.4 Struttura della Tesi

La tesi è strutturata come segue. Nel **Capitolo 2** vengono introdotti i concetti fondamentali di blockchain, Smart Contract e Mutation Testing. Il **Capitolo 3** presenta nel dettaglio la pipeline di generazione, illustrando i due approcci adottati per la generazione dei test case e il metodo adottato per la valutazione dei test generati. Il **Capitolo 4** presenta la metodologia sperimentale, i cui risultati sono discussi nel **Capitolo 5**. Infine, il **Capitolo 6** conclude il lavoro e delinea possibili sviluppi futuri.

CAPITOLO 2

Background

In questo capitolo vengono introdotte le basi concettuali utili per contestualizzare il lavoro di tesi.

2.1 Blockchain

La tecnologia blockchain ha rivoluzionato il mondo della gestione dei dati e delle transazioni digitali, trasferendo il meccanismo di fiducia dalle istituzioni centralizzate agli algoritmi crittografici. Le informazioni, a differenza dei database centralizzati dove la gestione dei dati è affidata a un singolo server centrale, sono conservate in un registro distribuito, condiviso e sincronizzato su una rete di nodi paritari (**peer-to-peer**). Questo approccio permette di superare le criticità dei sistemi attuali, spostando la fiducia “dalle istituzioni alla matematica (trusting math)” [1]. Viene chiamato **trustless** un sistema dove la correttezza delle transazioni è validata non da un'autorità, ma dalla verifica crittografica compiuta dal consenso della rete.

Una volta validate le transazioni, esse vengono raggruppate in una struttura dati che garantisce la sicurezza del registro: la **catena dei blocchi**. Ogni blocco contiene i dati delle transazioni e un timestamp¹ e l'elemento fondamentale: l'**hash del blocco**

¹l'ora esatta della creazione del blocco

precedente. A ciascun blocco viene inoltre assegnato un hash univoco, calcolato crittograficamente in funzione di tutte le informazioni contenute nel blocco. Di conseguenza, una semplice modifica ai dati interni cambierebbe interamente il suo hash, e poiché ogni blocco successivo include l'hash del blocco precedente, invaliderebbe a catena tutti gli hash successivi, rendendo la modifica evidente e rifiutata dalla rete. Questa architettura garantisce alla blockchain **immutabilità** e, parallelamente, con la distribuzione del registro su tutti i nodi, **trasparenza**, permettendo a qualsiasi nodo di verificare l'integrità della storia transazionale senza intermediari. Tale immutabilità rappresenta il vincolo più critico per lo sviluppo di Smart Contract e, in generale, di applicazioni legate alla blockchain. A differenza del software tradizionale, una volta pubblicato sulla Blockchain non può essere modificato o risolto facilmente; pertanto, ogni vulnerabilità diventa un rischio permanente.

Evoluzione da Bitcoin a Ethereum. La prima generazione di blockchain, rappresentata da Bitcoin [5], era concentrata principalmente sulle transazioni finanziarie peer-to-peer, con un linguaggio di scripting limitato. L'introduzione di **Ethereum** ha segnato un'evoluzione fondamentale. Progettata per estendere la capacità della tecnologia blockchain ben oltre la semplice valuta digitale, Ethereum [6] è una blockchain con un linguaggio di programmazione (Solidity) che consente a chiunque di poter scrivere Smart Contract e applicazioni decentralizzate (dApps). La sua innovazione principale è l'**Ethereum Virtual Machine (EVM)**, che consente l'esecuzione di logiche computazionali arbitrarie, inclusi cicli e stati complessi. Tuttavia, introduce una complessità algoritmica che aumenta il rischio di vulnerabilità, rendendo la verifica del software molto più problematica rispetto a semplici transazioni di pagamento.

2.2 Smart Contract su Ethereum

Nel 1994, il crittografo Nick Szabo [7] teorizzò il concetto di Smart Contract, definendolo come un insieme di protocolli utilizzati per eseguire promesse in formato digitale. Come detto in precedenza, il motivo alla base è rimuovere gli intermediari fiduciari, ridurre i costi ed eliminare l'obbligo di verifica. Szabo individua il **distributore automatico** come archetipo primitivo degli Smart Contract. Il funzionamento è

simile: la macchina incapsula le regole del contratto (“chi inserisce il gettone riceve la merce”) direttamente nell’hardware, rendendole, appunto, inviolabili. L’hardware, infatti, agisce sia da esecutore che da custode del valore, garantendo a chiunque soddisfi le condizioni di input di ricevere l’output (in modo deterministico) desiderato, rendendo la violazione del contratto economicamente svantaggiosa.

Ethereum implementa questa logica contrattuale attraverso la **Ethereum Virtual Machine (EVM)**, un ambiente di esecuzione distribuito, isolato (Sandbox) e quasi Turing-completo², dove ogni nodo della rete esegue le stesse istruzioni per raggiungere il consenso sullo stato globale della rete.

Il ciclo di vita di uno Smart Contract [6, 8] segue fasi rigide: una volta scritto il codice in un linguaggio ad alto livello (tipicamente Solidity), viene compilato e generato il **Bytecode**, il linguaggio macchina a basso livello dell’EVM. Allo stesso tempo viene generata anche l’**ABI (Application Binary Interface)**, un’interfaccia JSON necessaria per codificare e decodificare le chiamate alle funzioni del contratto. Successivamente, il bytecode viene deployato sulla blockchain tramite una transazione di creazione.

Arrivata la conferma, il codice viene registrato in modo permanente ad un indirizzo specifico e da questo momento la sua logica diventa immutabile e non può essere modificata. Infine, ogni interazione con il contratto avvia l’esecuzione del bytecode. Come detto in precedenza, l’EVM è Turing-completa, quindi esiste il rischio teorico di loop infiniti, che potrebbero bloccare la rete. Viene introdotto il concetto di **Gas**, un’unità di misura che assegna un costo computazionale a ogni istruzione. L’emittente della transazione dovrà quindi pagare una **fee** proporzionata al gas consumato. Nel caso in cui il gas terminasse prima della fine dell’esecuzione (Out of Gas), la transazione fallisce e tutte le modifiche allo stato vengono ripristinate (revert), garantendo l’integrità.

2.3 Vulnerabilità negli Smart Contract

La classificazione delle vulnerabilità negli Smart Contract è supportata da standard industriali come **SWC Registry** [9] e **OWASP Smart Contract Top 10** [10]. Una

²Indica che il sistema può essere programmato per eseguire qualsiasi logica complessa, inclusi i cicli, può quindi risolvere qualsiasi problema computazionale risolvibile.

recente revisione della letteratura condotta da Iuliano et al. [4] ha proposto una tassonomia aggiornata, confermando che le criticità principali risiedono nella logica applicativa, ossia in quella scritta dagli sviluppatori. Di seguito si riporta un'analisi delle criticità logiche più ricorrenti che i test automatizzati devono intercettare.

Reentrancy [9]. Si verifica quando uno Smart Contract cede temporaneamente il flusso di controllo a un contratto esterno, ad esempio durante l'invio di Ether, prima di aver completato l'aggiornamento del proprio stato interno. La vulnerabilità sfrutta il fatto che, alla ricezione di fondi o di una chiamata, un contratto può eseguire codice arbitrario. Nel momento in cui l'attaccante ottiene il controllo, invoca nuovamente la funzione originale del contratto vittima. Poiché la prima esecuzione non è ancora terminata ma è in attesa, le modifiche allo stato (come la deduzione del saldo dell'utente) non sono ancora avvenute; di conseguenza, i controlli di validità hanno nuovamente successo.

Integer Overflow e Underflow [9]. Questa criticità è legata alla gestione delle variabili, che hanno capacità di memoria limitata. Se non gestita correttamente, un'operazione che produce un risultato eccedente i limiti del tipo di dato (ad esempio `uint256`) causa un "reset" del valore. Storicamente era necessario ricorrere a librerie esterne per prevenire questo fenomeno; le versioni moderne di Solidity includono controlli nativi che, in caso di errore, inviano la transazione in *revert*. La vulnerabilità rimane tuttavia rilevante per due motivi: la presenza di codice legacy e l'uso intenzionale di blocchi `unchecked` per ottimizzare il Gas, in cui i controlli di sicurezza vengono disattivati. Una gestione errata può permettere a un attaccante di manipolare i bilanci.

Timestamp Dependency [9]. Accade quando i contratti utilizzano l'orario del blocco (timestamp) per gestire le scadenze o come sorgente di entropia per generare numeri casuali. Tuttavia, l'orario esatto è determinato dai nodi che validano i blocchi, i quali possono manipolarne il valore a proprio vantaggio, compromettendo l'integrità della logica temporale.

Unchecked External Calls [9]. Si verifica quando uno Smart Contract invia fondi o invoca un altro Smart Contract tramite primitive di basso livello. Queste operazioni possono fallire restituendo `false` senza interrompere l'esecuzione con un'eccezione. È imperativo, quindi, che lo sviluppatore verifichi esplicitamente l'esito dell'operazione per evitare stati inconsistenti.

2.4 Software Testing e Mutation Testing

Il testing deve garantire standard di correttezza e sicurezza elevati prima del deployment in rete. Tra le varie tecniche, il **testing di unità** mira ad isolare le singole unità di codice (tipicamente le funzioni del contratto) per verificare il loro comportamento a fronte di input deterministici, controllando che l'output prodotto o la modifica di stato corrisponda ai risultati attesi. L'insieme di tali test costituisce la **test suite**, progettata per garantire una copertura esaustiva dei requisiti funzionali. È importante introdurre una metrica fondamentale: il **test pass rate** (spesso chiamato *code correctness* [8]) rappresenta la percentuale di unit test superati con successo rispetto al totale dei test eseguiti. Per automatizzare questo processo, l'ecosistema Ethereum si affida a dei framework di sviluppo come **Hardhat** [11], che consente di compilare i contratti e di deployarli su una rete locale effimera (Hardhat Network) che simula fedelmente il comportamento dell'EVM ed esegue i test in *TypeScript*. Tale configurazione permette di simulare scenari complessi, inclusi il revert delle transazioni e la manipolazione del timestamp, in un ambiente di esecuzione controllato.

La metrica più diffusa per valutare la copertura del testing è la **code coverage** (copertura del codice) che misura la percentuale di istruzioni, rami decisionali o funzioni del codice sorgente che vengono fisicamente eseguiti durante il lancio dei test automatizzati. La copertura è una condizione necessaria, ma non sufficiente, per garantire la robustezza del codice [4]. Studi recenti dimostrano infatti che il limite della code coverage risiede nell'incapacità di verificare effettivamente la correttezza logica del test. Di conseguenza, affidarsi unicamente ad essa può generare una falsa percezione di sicurezza e lasciare il contratto esposto a difetti critici non rilevati.

Mutation Testing. Per superare il limite della code coverage, la letteratura accademica propone il mutation testing [12], una tecnica di testing del software basata sull'iniezione deliberata di difetti. Il principio di base del mutation testing è quello di simulare errori di programmazione comuni per valutare l'effettiva capacità della test suite; il processo richiede più fasi [4]. Nella prima fase, a partire dal codice sorgente originale, vengono create molteplici versioni modificate, dette **mutanti**. Questi ultimi presentano una piccola alterazione sintattica, ad esempio sostituendo l'operatore di addizione + con quello di sottrazione -. Queste alterazioni sono introdotte da regole predefinite, implementate tramite **operatori di mutazione**. Nella seconda fase la test suite viene eseguita per ogni mutante. L'ultima fase riguarda l'analisi del risultato: se i test falliscono, significa che hanno rilevato l'anomalia introdotta, e il mutante viene detto **ucciso** (killed). Se i test passano nonostante la modifica, significa che non sono stati in grado di distinguere il codice corretto da quello errato; il mutante in questo caso viene chiamato **sopravvissuto** (survived). Viene assegnato un punteggio alla test suite, chiamato **mutation score**:

$$Mutation\ Score = \frac{Killed\ Mutants}{Total\ Mutants - Equivalent\ Mutants} \times 100$$

dove **killed mutants** indica il numero di mutanti uccisi, ossia la mutazione ha causato il fallimento di almeno un test, invece **equivalent mutants** indica il numero di mutanti sintatticamente diversi dall'originale, ma con lo stesso comportamento.

L'applicazione del mutation testing sugli Smart Contract richiede un adattamento alla specificità del linguaggio e dell'architettura blockchain, come i difetti legati al *Gas* e alla gestione degli indirizzi e alle transazioni. Esistono degli operatori di mutazione ad hoc per simulare le vulnerabilità tipiche della blockchain, come ad esempio: la rimozione di controlli di sicurezza (`require`), manipolazione delle variabili globali, o l'alterazione della visibilità delle funzioni. Diversi strumenti sono stati sviluppati per automatizzare questo processo, ad esempio SuMo [4] che integra un ampio set di operatori specifici per Solidity.

2.5 Large Language Models & Code Generation

L'ultimo decennio ha visto una rapida evoluzione nel campo dell'elaborazione del linguaggio naturale (NLP), culminata con l'avvento dei **Large Language Models (LLM)**. Questi modelli, addestrati su vastissimi corpus di dati, hanno dimostrato capacità emergenti che vanno oltre la semplice comprensione del testo umano, grazie alla loro abilità di apprendere pattern sintattici e semantici complessi, gli LLM si sono affermati come strumenti fondamentali anche nell'ambito della **code generation** [2], permettendo l'automazione della scrittura di codice software sintatticamente corretto a partire da descrizioni in linguaggio naturale.

Alla base di questa rivoluzione vi è l'architettura **Transformer**, introdotta per gestire sequenze di dati tramite meccanismi di *Self-attention*. Nel contesto della programmazione, questi modelli trattano il codice sorgente come una sequenza di token, apprendendone la sintassi rigida e le dipendenze strutturali. Nell'ambito specifico della Code Generation, la letteratura identifica nei modelli **Decoder-Only** l'architettura di riferimento [2]. Questi modelli sono addestrati con l'obiettivo specifico di predire il token successivo in una sequenza (*next-token prediction*). Tale caratteristica li rende particolarmente efficaci nella generazione di codice, mantenendo un'elevata coerenza logica e sintattica. Tuttavia, non è chiaro se questi modelli siano in grado di generare suite di test non solo compilabili, ma anche efficaci.

CAPITOLO 3

Metodologia Sperimentale

L'obiettivo primario di questo lavoro è valutare le capacità e i limiti operativi dei Large Language Models (LLM) nella generazione automatizzata di test suite per Smart Contract. Sono stati utilizzati due approcci differenti. Il primo si basa sull'approccio **Zero-Shot**, che funge da baseline per l'esperimento: in questo scenario, il modello viene interrogato con un prompt diretto e semplice (ad esempio: *"Scrivi una test suite hardhat per questo contratto"*), senza fornire esempi pregressi né feedback sugli errori. Il secondo approccio è **Iterativo** e introduce un'architettura più complessa basata su tre iterazioni. In questo caso, il sistema verifica la qualità del test generato e, qualora si verifichi un errore in fase di compilazione o a runtime, cattura il log dell'errore e lo inietta nel prompt di retry, insieme al codice della versione fallita e al contratto originale. Il modello dispone di fino a tre tentativi per analizzare l'errore, autocorreggersi e produrre una soluzione funzionante.

3.1 LLM & Dataset

Il nucleo generativo del sistema è costituito dal modello **Qwen 2.5-coder:32B**. La scelta di questo modello Open-Weights è motivata dalle sue eccellenti capacità di ragionamento e dalla gestione efficace di istruzioni complesse e contesti estesi, come

documentato da Hui et al. [13]. Date le dimensioni del modello e l'elevata richiesta di risorse computazionali, è stato necessario disaccoppiare l'architettura del sistema: il motore di generazione è stato ospitato su un'istanza cloud **Google Colab Pro**, equipaggiata con acceleratore grafico NVIDIA A100 (40GB VRAM), dove il modello è stato servito tramite framework Ollama, configurato come server API per accettare le richieste remote. La logica di validazione è stata invece mantenuta in ambiente locale; per collegare i due ambienti è stato configurato un tunnel crittografico tramite **Ngrok**, consentendo allo script locale di inviare prompt complessi al server cloud e di ricevere il codice generato in tempo reale.

Il dataset è formato da contratti basati sull'ultima versione di Solidity, estratti dalla blockchain di Ethereum (Maggio 2025). Poiché la raccolta è iniziata dall'ultimo blocco disponibile, i contratti selezionati rappresentano il codice più recente in rete. Inoltre, è stata effettuata una fase preliminare di filtraggio poiché non tutti i contratti del dataset originale risultavano idonei alla generazione dei test. Sono stati esclusi dal campione i file non contrattuali (come interfacce pure o librerie astratte prive di logica testabile), i contratti basati su versioni di Solidity obsolete o incompatibili e i contratti con dipendenze irrisolvibili. Il dataset è stato infine suddiviso in tre categorie dimensionali: **Small, Medium, Large**. L'analisi si è concentrata prevalentemente sulle categorie Small e Medium. L'intera pipeline di generazione e validazione è disponibile nella repository GitHub¹.

3.2 Approccio Zero-shot

L'approccio Zero-Shot è stato adottato come *baseline* sperimentale per valutare le capacità naive del modello. La strategia si è basata sull'utilizzo di un prompt generico (come mostrato nel Listing 3.1), formulato per chiedere la generazione di una test suite Hardhat senza imporre vincoli architetturali specifici né fornire esempi. Il processo è stato rigorosamente limitato a un singolo tentativo di generazione per ogni contratto, escludendo a priori qualsiasi meccanismo di autocorrezione o di feedback sugli errori. Le limitazioni strutturali emerse in questa fase, la cui analisi

¹<https://github.com/FaiellaFrancesco/Tirocinio-Smart-Contract>

dettagliata è rinviata al capitolo dei Risultati, hanno reso necessaria la transizione all'architettura iterativa sviluppata nella fase successiva.

```

1 You are an experienced Smart Contract Auditor and Solidity Developer.
2
3 Write a complete Hardhat test suite in TypeScript for the following Solidity smart contract
4 ```solidity
5 [INSERT SMART CONTRACT CODE HERE]

```

Listing 3.1: Prompt Zero-Shot

3.3 Approccio Iterativo

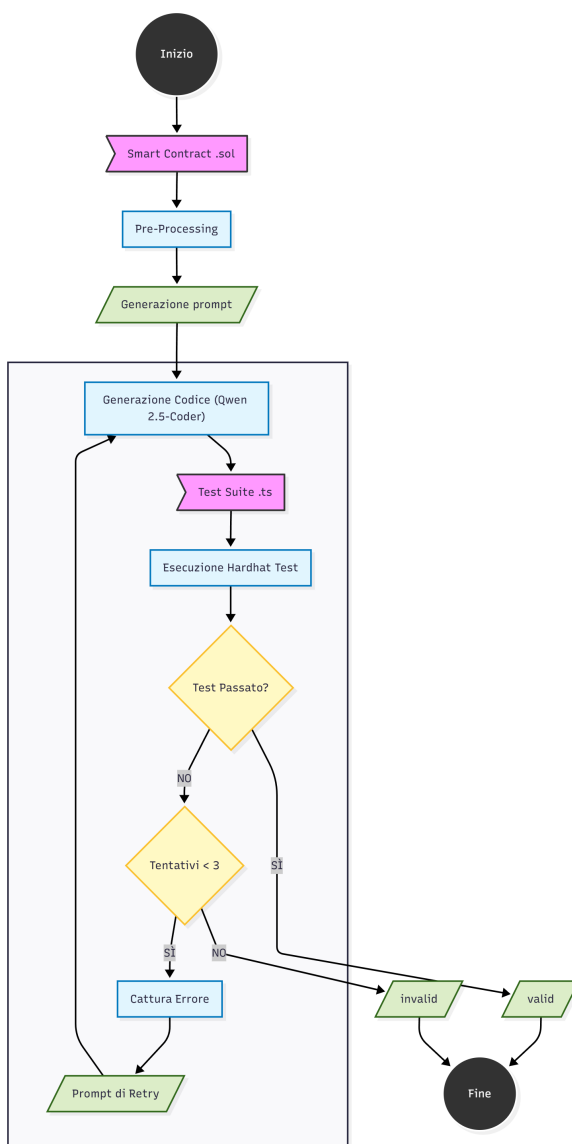


Figura 3.1: Flusso logico della pipeline iterativa con meccanismo di Self-Healing.

Come illustrato nella figura 3.1, l'architettura implementata abbandona la linearità dei processi di generazione tradizionali e adotta un flusso iterativo, governato dalla validazione a runtime. Il ciclo di vita si articola in diversi passaggi.

Input: Il codice sorgente dello Smart Contract e i metadati estratti vengono assemblati in un prompt iniziale, ottimizzato per fornire al modello tutte le informazioni contestuali necessarie al primo tentativo, come mostrato nel Listing 3.2.

```

1 You are an Elite Smart Contract Auditor and QA Engineer.
2 Your task is to write a production-grade, bug-free TypeScript test suite for a Solidity
   smart contract using the Hardhat framework.
3
4 ### 1. TECHNICAL STACK & CONSTRAINTS
5 * Target Framework: Hardhat (Standard) + Mocha + Chai.
6 * Ethers.js Version: v5 (Strictly enforce v5 syntax).
7 * Solidity Version: {SOL_VERSION}
8
9 ### 2. CRITICAL ENVIRONMENT RULES (MUST FOLLOW)
10 The execution environment is strict. Adhere to these anti-patterns:
11 * NO Waffle Matchers:
12   * BAD: await expect(tx).to.emit(...) or to.be.revertedWith(...)
13   * GOOD (Reverts): Use standard try/catch blocks.
14   * GOOD (Events): Use receipt.events.find(...) or verify state changes.
15 * BigInt Compatibility:
16   * Hardhat/Ethers v5 returns BigNumber. TypeScript uses native BigInt.
17   * GOLDEN RULE: Always .toString() both sides before comparing with .equal().
18 * Artifact Resolution (HH700):
19   * You MUST use this exact Fully Qualified Name (FQN): "{FQN_LABEL}"
20
21 ### 3. DEPLOYMENT STRATEGY
22 * Use loadFixture from @nomicfoundation/hardhat-network-helpers.
23 * Constructor Analysis: Params required: {CTOR_PARAMS}
24
25 ### 4. TEST COVERAGE REQUIREMENTS
26 Structure your test suite with these describe blocks:
27 1. Deployment: Verify initial state variables.
28 2. Core Logic (Happy Path): Test every public/external function.
29   * Logic Check: If using allowance, call approve BEFORE transferFrom.
30 3. Access Control & Security: Test onlyOwner modifiers if present.
31 4. Edge Cases & Reverts: Test conditions that trigger require statements.
32
33 ### 5. SOURCE CODE ANALYSIS
34 Analyze the following contract:
35 ```solidity
36 {CONTRACT}

```

Listing 3.2: Template del Prompt "Elite Auditor" utilizzato nella pipeline iterativa.

Generazione ed Esecuzione: Il modello genera una prima bozza della test suite, ma a differenza di approcci statici, il sistema non considera l'output come definitivo, ma lo sottopone immediatamente ad una **validazione dinamica** con Hardhat in un ambiente isolato. Se la validazione ha successo, il ciclo si interrompe immediatamente, il file viene marcato come valido e salvato nella directory di output. Se Hardhat restituisce un errore, il sistema attiva il meccanismo di **Self-Healing**. Quest'ultimo cattura lo *standard error* e verifica il contatore dei tentativi. Se il numero di iterazione è inferiore alla soglia massima ($MAX_ATTEMPTS = 3$), l'errore viene iniettato nel **prompt di retry**. Questo nuovo prompt contiene il codice generato nel try precedente, il messaggio di errore esatto, il contratto iniziale e delle euristiche di correzione pertinenti, istruendo il modello a riparare la propria soluzione. Il ciclo continua fino al successo della validazione o all'esaurimento dei tentativi disponibili. In quest'ultimo caso, vengono salvati i log per un'analisi post-mortem.

Le euristiche integrate nel prompt di retry non sono predefinite, ma sono state ricavate progressivamente attraverso un'analisi empirica dei log di errore. Una prima serie di regole è stata derivata dai fallimenti sistematici osservati nella fase Zero-Shot; successivamente, ulteriori euristiche sono emerse dall'osservazione iterativa dei log prodotti nei tentativi falliti dell'approccio Iterativo. In questo modo, l'architettura si è evoluta secondo un processo di raffinamento incrementale, guidato dai pattern ricorrenti di errore rilevati durante l'esecuzione reale dei test.

3.4 Valutazione dei Empirica dei Test Generati

In una prima istanza, la valutazione si concentra sulla capacità della pipeline di produrre codice "funzionante". Ai fini di questo studio, una test suite è definita tale se soddisfa simultaneamente due condizioni: (i) **compilabilità**: il codice TypeScript generato deve essere sintatticamente corretto e privo di errori che ne impediscono la compilazione e (ii) **eseguibilità**: il test deve poter essere avviato ed eseguito completamente nell'ambiente Hardhat senza interruzioni anomale, terminando il processo con esito positivo (`exit 0`). L'analisi comparativa tra i due approcci sperimentati (Zero-Shot e Iterativo) è stata condotta applicando queste metriche ai contratti appar-

tenenti alle categorie dimensionali **Small** e **Medium**, al fine di evidenziare l’impatto della complessità del codice sorgente sulle prestazioni di generazione.

In seguito, i contratti sono stati analizzati tramite mutation testing. Preliminarmente, è stato necessario isolare l’ambiente di validazione per evitare che i mutanti generati da **SuMo** sovrascrivessero i contratti originali necessari alle altre fasi. Per quanto riguarda la strategia di mutazione, sono stati utilizzati tutti gli operatori disponibili, al fine di evitare qualsiasi *bias di selezione*. Tale configurazione comprende sia mutazioni sintattiche tradizionali che gli operatori specifici per il dominio Ethereum, garantendo così che il mutation score finale rifletta la capacità della test suite di affrontare le vulnerabilità introdotte.

L’esecuzione di SuMo è stata applicata rigorosamente al sottoinsieme di contratti per i quali la pipeline ha generato una test suite sintatticamente valida e funzionante, un requisito fondamentale per l’utilizzo del tool. Operativamente, per ogni contratto target, lo strumento analizza l’Abstract Syntax Tree (AST) e istanzia l’insieme completo dei mutanti applicabili in base agli operatori scelti. Successivamente, il sistema avvia una sessione di testing isolata per ciascun mutante: la test suite generata dall’LLM viene eseguita sul codice mutato e l’esito viene registrato.

Se la test suite **fallisce**, ossia rileva l’errore, il mutante viene marcato come *Killed*. Viceversa, se tutti i test **passano** nonostante la mutazione, il mutante è considerato *Survived*, che indica quindi una carenza nella copertura logica dei test. Al termine del processo, viene calcolato il mutation score (già definito nella sezione 2.4). Riguardo ai mutanti equivalenti, ovvero mutazioni che non alterano la semantica del programma, la loro identificazione non è automatica, e la verifica manuale su larga scala risulta impraticabile in termini di costi temporali; si assume quindi un valore pari a zero ($E = 0$). Di conseguenza, il punteggio ottenuto rappresenta una stima *lower bound* della robustezza reale.

CAPITOLO 4

Analisi dei Risultati

Il capitolo è dedicato all'analisi critica dei risultati ottenuti attraverso la pipeline di generazione. La valutazione della qualità del codice si basa su due qualità fondamentali: la **validità sintattica** e la **validità semantica**. Di seguito sono presentati i dati raccolti, con un'analisi dei principali errori.

Risultati Zero-Shot. L'approccio preliminare Zero-Shot ha registrato un tasso di successo nullo, evidenziando l'incapacità del modello di gestire autonomamente le dipendenze ambientali in assenza di un contesto strutturato. Tuttavia, l'analisi sistematica dei log di errore è stata determinante per formalizzare le euristiche di correzione, che hanno reso possibile il successo della fase iterativa.

Risultati Approccio Iterativo. L'introduzione dell'approccio iterativo e delle euristiche di correzione ha segnato un netto cambio di passo: partendo da 291 contratti (categoria **Small**) a logica lineare, la pipeline è riuscita a produrre 87 test suite funzionanti, con un tasso di successo del $\approx 30\%$. L'aumento di complessità dei contratti ha causato un drastico calo delle prestazioni. Su 38 contratti (Categoria **Medium**), solo 3 test suite hanno superato la validazione, pari a circa 8%.

Tabella 4.1: Risultati dell’Approccio Iterativo suddivisi per complessità dei contratti.

Categoria	Contratti Totali	Test Suite Valide	Tasso di Successo	Mutation Score
Small	291	87	$\approx 30\%$	63.5%
Medium	38	3	$\approx 8\%$	0%

4.1 Risultati sulla generazione dei test case

Di seguito, si riportano i risultati relativi alla compilabilità e all’eseguibilità dei casi di test generati con i due approcci proposti.

4.1.1 Analisi dei Fallimenti della Fase Zero-Shot

I risultati evidenziano un divario prestazionale netto tra i due approcci. La strategia **Zero-Shot**, basata su un singolo prompt privo di contesto, ha registrato un tasso di fallimento del 100%. Sebbene i test generati apparissero sintatticamente plausibili a una prima analisi superficiale, l’esecuzione falliva sistematicamente sia in fase di pre-processing sia in fase di runtime. L’analisi approfondita dei log conferma che tale insuccesso non è imputabile a una carenza intrinseca del modello nella scrittura della logica di test, bensì alla difficoltà di inferire correttamente la configurazione e i vincoli dell’ambiente di esecuzione. Nello specifico, è stato possibile categorizzare le cause di fallimento in due macroaree critiche.

Cecità Ambientale. Il modello tendeva a generare codice dipendente da plugin o librerie non installate nell’infrastruttura locale, ad esempio `ethereum-waffle`. Inoltre, si è riscontrato un uso frequente di sintassi incompatibili, caratterizzato da un mix incoerente tra versioni diverse delle librerie (es. l’importazione di metodi tipici di *Ethers v6* in un ambiente configurato per la *v5*), che ha portato a errori di runtime immediati. Non avendo accesso ai contenuti dei file importati, il modello non riusciva a ricostruire l’albero di ereditarietà del contratto. Di conseguenza, “allucinava” chiamate a funzioni inesistenti o tentava di testare metodi con visibilità interna/privata non accessibili dai test, causando errori bloccanti in fase di compilazione.

Ambiguità degli Artefatti. Il framework Hardhat falliva spesso nel collegare il test generato al contratto target corretto a causa di collisioni di nome all'interno del dataset (es. più contratti chiamati `Token`). Questa ambiguità nella risoluzione degli artefatti generava errori di tipo `HH700` o `HH701`, impedendo il deployment della fixture di test.

4.1.2 Analisi dei Fallimenti Residui nell'Approccio Iterativo

Durante la fase iterativa è stata inizialmente osservata la persistenza di errori ricorrenti. L'approccio adottato non si è limitato alla registrazione dei fallimenti, bensì si è evoluto in un processo di raffinamento iterativo; l'analisi dei log degli errori ha guidato l'aggiornamento e l'ottimizzazione delle euristiche di correzione integrate nel prompt. Questa fase di raffinamento non è stata limitata alla sola analisi preliminare dei fallimenti Zero-Shot, ma è proseguita durante l'intero processo iterativo; ogni nuova classe di errore osservata nei tentativi falliti è stata trasformata in un'ulteriore euristica. Nonostante l'efficacia delle euristiche nel risolvere gli errori sintattici, l'analisi dei log post-mortem delle test suite rimaste invalide (dopo i 3 tentativi) ha evidenziato limiti strutturali nella capacità di autocorrezione del modello. I fallimenti residui si concentrano in tre categorie distinte, di seguito analizzate.

Persistenza nell'Allucinazione. In alcuni casi, il modello entra in un ciclo di correzione inefficace, dimostrando una resistenza a seguire le istruzioni negative. Un esempio tipico si verifica di fronte all'errore *"function not found"*: nonostante l'euristica suggerisca esplicitamente di rimuovere il test case, l'LLM tenta ostinatamente di "riparare" la chiamata, ad esempio inventando parametri diversi o provando a istanziare un *mock* della funzione inesistente. Questo comportamento indica che, a causa del bias statistico, il modello tende a ignorare i vincoli del prompt di correzione.

Complessità delle Dipendenze Esterne. Alcuni contratti presentano dipendenze implicite da interfacce o librerie infrastrutturali (es. Oracoli o DEX come Uniswap) che richiedono un setup di *mocking* avanzato nell'ambiente di test. Anche se il codice di queste interfacce viene iniettato correttamente nel prompt (Context Injection), l'LLM spesso fallisce nel generare la complessa logica di setup necessaria a simulare questi

componenti on-chain nel `beforeEach`, portando a errori di esecuzione irrisolvibili (es. chiamate a indirizzi vuoti).

Errori Semantici e di Business Logic. Mentre le euristiche si dimostrano eccellenti nel risolvere errori sintattici, risultano meno efficaci sugli errori logici. Quando l’LLM non comprende appieno la logica di business del contratto (es. un meccanismo di tassazione o di staking), genera asserzioni matematiche errate (`‘AssertionError’`). In questo scenario, l’approccio iterativo non riesce a convergere. Il modello mira ad apportare correzioni sintattiche a un codice logicamente fallace. Poiché il problema non risiede nella forma, bensì nella comprensione del contratto stesso, il modello non riesce a costruire test che coprano le logiche di business non tradizionali.

4.2 Risultati del mutation testing

La seconda analisi della fase si concentra sulla qualità semantica delle test suite generate funzionanti; la domanda non è più: *“il test funziona?”*, ma diventa: *“il test è robusto?”*. Sono stati generati in totale 6920 mutanti, per 87 test suite della categoria Small, registrando un mutation score medio del 63.5%. Da un’analisi approfondita, possiamo intuire come l’LLM riesca a generare test efficaci per la logica di base e le funzioni standard, ma non copre scenari di confine (edge cases). A supporto di ciò, analizziamo i due casi limite di questa categoria.

Best case. Il contratto `EscrowService` ha ottenuto un Mutation Score del 85%. Il modello ha replicato pattern di test noti, garantendo una copertura efficace sia della logica di business sia della sicurezza di base. Il tool SuMo ha generato un mutante del contratto rimuovendo il modificatore di sicurezza `onlyBuyer` da una funzione critica incaricata del rilascio dei fondi, rendendo la funzione accessibile a chiunque e eliminando la protezione originale. La test suite generata dall’LLM si è dimostrata in grado di rilevare questo grave difetto di sicurezza. Il test era stato progettato per provare a chiamare quella funzione con un account non autorizzato, con l’aspettativa che la transazione fallisse. Poiché il mutante aveva rimosso la protezione, la transazione tentata nel test ha avuto successo; di conseguenza, l’asserzione del test (che si

aspettava un errore) è fallita, portando all'identificazione e all'eliminazione del mutante. Questo esempio dimostra che il modello ha correttamente inferito la necessità di testare non solo il funzionamento corretto, ma anche i vincoli di sicurezza.

Worst case. La performance peggiore si verifica in un contesto di contratto che prevede logiche di business custom, un'architettura non standard e meccanismi di tassazione proprietaria. Questo conferma che la capacità di ragionamento del modello degrada drasticamente quando non può affidarsi alla "memoria" di pattern visti nel training set, ma deve dedurre test case per logiche mai viste prima. Durante il test, il tool ha applicato l'operatore **Integer Literal Replacement (ILR)**, modificando una costante numerica critica nel codice; ad esempio, azzerando la percentuale di commissione o modificando la total supply iniziale. La test suite generata dall'LLM, pur eseguendo correttamente le funzioni di trasferimento, non conteneva asserzioni specifiche sui saldi finali; il test si limitava a verificare che la transazione non fallisse (ossia non andasse in revert), senza controllare matematicamente che l'importo arrivato a destinazione fosse corretto a netto di tasse. Il mutante che ha alterato la matematica interna è sopravvissuto indisturbato, dimostrando che, di fronte a logiche non standard, l'LLM tende a generare test "superficiali" che validano l'esecuzione della funzione ma ignorano completamente la correttezza dei dati processati.

4.2.1 Analisi dei Mutanti Sopravvissuti

L'analisi qualitativa dei log di SuMo ha permesso di isolare le categorie specifiche di mutanti che sfuggono sistematicamente alla verifica delle test suite generate.

Cecità agli Eventi (Operator EED). Una percentuale significativa di mutanti legati alla rimozione dell'emissione di eventi (`Event Emission Deletion - EED [14]`) è sopravvissuta. Questo dato evidenzia una lacuna specifica nella strategia di testing del modello: sebbene le funzioni vengano invocate correttamente, si generano raramente asserzioni che verifichino la presenza e la correttezza dei log della transazione. Tale omissione è critica in ambito blockchain, dove gli eventi rappresentano l'unico meccanismo di comunicazione osservabile dalle applicazioni off-chain.

Gestione superficiale dei Trasferimenti (Operator ETR). Anche nei casi di successo, i mutanti di tipo `Ether Transfer Replacement` – ETR [14] sono sopravvissuti con tassi vicini al 100%. Ciò suggerisce che le test suite si limitano a verificare lo stato finale (il saldo), senza validare la correttezza del meccanismo di trasferimento sottostante (es. l'uso di `call` rispetto a `transfer`). Questa superficialità lascia il contratto esposto a rischi di sicurezza legati alla gestione del gas o alla verifica dei valori di ritorno.

Resilienza sulla logica pura. Il modello si dimostra efficace nel rilevare alterazioni alla logica aritmetica (BOR) e relazionale (ROR). Il fatto che questi mutanti vengano regolarmente uccisi conferma che le capacità di ragionamento matematico di *Qwen 2.5-Coder* sono solide. Tuttavia, il contrasto con i punti precedenti indica che al modello manca ancora la piena “consapevolezza architetturale” necessaria a validare la semantica profonda e specifica di uno Smart Contract.

Mutation Testing sulla categoria Medium. Un'attenzione particolare merita l'analisi condotta sui 3 contratti della categoria **Medium** che hanno superato la fase di validazione sintattica. Su questi contratti, SuMo ha generato **344 mutanti** con un **mutation score dello 0%**. Un valore così basso indica che nessun mutante generato è stato rilevato dalla test suite, il che evidenzia la scarsa robustezza dei test case.

CAPITOLO 5

Discussione

La pipeline iterativa ha dimostrato che l’LLM agisce in modo efficiente per la logica standard, permettendo al programmatore di concentrarsi esclusivamente sugli scenari critici e sulle logiche di sicurezza (edge cases) che l’LLM tende a trascurare. Tuttavia, l’analisi semantica ha rivelato che il modello tende a ottimizzare la **plausibilità sintattica** anziché la correttezza logica profonda. Questo fenomeno è evidente nel contrasto tra l’elevato successo sui mutanti matematici e il fallimento sui mutanti legati agli eventi e ai trasferimenti, che richiedono una comprensione del contesto e degli effetti collaterali della transazione.

Il dato più critico emerso è il mutation score dello 0% registrato sugli Smart Contract della categoria **Medium**. Questo risultato indica una scarsa capacità di ragionamento all’aumentare della complessità strutturale. L’LLM, infatti, per soddisfare i vincoli di compilazione in contesti con dipendenze complesse, generava test che invocavano le funzioni, assicurandosi solo che non andassero in errore, omettendo però tutte le asserzioni di stato necessarie. Il test risulta quindi valido dal punto di vista sintattico, ma, semanticamente, errato, incapace di rilevare qualsiasi mutazione. Ciò conferma che, oltre una certa soglia, il modello perde la coerenza necessaria.

Confronto con lo Stato dell'Arte. È fondamentale contestualizzare questi risultati rispetto allo stato dell'arte. Sebbene questo studio si sia concentrato su un modello Open-Weights accessibile localmente, la letteratura indica che modelli proprietari, come **GPT-4o (OpenAI)** o **Gemini 2.5 Pro (Google)**, hanno capacità di *reasoning* significativamente superiori rispetto ai modelli aperti attuali. Uno studio del 2025 condotto da Xiao et al. [15] ha dimostrato che questi modelli di frontiera sono in grado di generare automaticamente exploit (codice d'attacco) funzionanti con un tasso di successo superiore al 90%, superando nettamente le controparti open weight. Questo risultato evidenzia una comprensione profonda delle vulnerabilità e della logica degli Smart Contract; è ragionevole ipotizzare che tale capacità di ragionamento superiore si rifletta anche nella generazione di test unitari più robusti, in grado di coprire gli scenari limite (*Edge Cases*) che i modelli attuali trascurano.

CAPITOLO 6

Conclusioni

Questo lavoro di tesi si è posto l’obiettivo di esplorare le potenzialità e i limiti dell’Intelligenza Artificiale Generativa nell’automazione del testing dei Smart Contract, un dominio caratterizzato da vincoli di sicurezza critici e dall’immutabilità del codice dispiegato. L’indagine ha seguito un percorso incrementale, partendo dall’analisi delle capacità *out-of-the-box* dei moderni Large Language Models *Open-Weights* (nello specifico **Qwen 2.5-Coder 32B**) fino alla definizione di un’architettura software in grado di mitigarne le carenze strutturali.

La fase sperimentale iniziale ha evidenziato che l’approccio *Zero-Shot* è insufficiente a gestire la complessità legata allo sviluppo degli smart contract. I modelli, pur dimostrando competenza nella sintassi di base, soffrono di una marcata “cecità ambientale”, fallendo nel risolvere dipendenze locali, versioni delle librerie e configurazioni specifiche del framework Hardhat. Per superare questa barriera sintattica, che impediva l’esecuzione di qualsiasi test, è stata progettata e implementata una **Pipeline Iterativa**. Questa evoluzione architetturale ha permesso di generare **87 suite di test valide e funzionanti**. Un ruolo centrale in questo risultato è stato svolto dalla progressiva definizione delle euristiche di correzione, derivate dallo studio sistematico dei log di errore sia nella fase *Zero-Shot* sia durante i tentativi falliti dell’approccio iterativo, creando la base necessaria per la successiva fase di valutazione qualitativa.

Il mutation testing ha tuttavia rivelato la natura duale dei risultati ottenuti. Se da un lato la pipeline ha garantito la validità sintattica, dall'altro la validità semantica ha mostrato limiti significativi, registrando un mutation score del 63.5% sui contratti di categoria *Small*. Il modello si è dimostrato efficace nel testare la logica matematica, ma carente nella verifica della logica di business e degli effetti collaterali; inoltre, si è osservata una scarsa capacità di ragionamento con un mutation score dello 0% sui contratti più complessi. L'analisi ha confermato che il modello tende a generare test che superano l'esecuzione, ma spesso mancano di asserzioni di stato valide.

Alla luce dei risultati ottenuti, l'adozione di questi strumenti non deve essere intesa come sostituzione dell'esperto umano, bensì come un passaggio verso un'**ingegneria assistita**, in cui la supervisione rimane un requisito imprescindibile per la sicurezza. La naturale evoluzione di questo lavoro di ricerca consiste nel **Fine-Tuning Supervisionato (SFT)**. L'obiettivo è creare un modello specializzato nel testing degli Smart Contract che non richieda euristiche esterne per generare codice corretto. In conclusione, il presente lavoro ha dimostrato come guidare un modello di intelligenza artificiale pre-trained nell'esecuzione di un compito specifico. I lavori futuri dovranno concentrarsi sul costruire un'intelligenza artificiale specializzata, trasformando le regole euristiche in intuizioni native del modello.

Bibliografia

- [1] M. Nofer, P. Gomber, O. Hinz, and D. Schiereck, "Blockchain," *Business & Information Systems Engineering*, vol. 59, no. 3, pp. 183–187, Jun 2017. [Online]. Available: <https://doi.org/10.1007/s12599-017-0467-3> (Citato alle pagine 1 e 4)
- [2] G. De Vito, D. D'Amici, F. Izzo, F. Ferrucci, and D. Di Nucci, "Llm-based generation of solidity smart contracts from system requirements in natural language: The astrakode case," in *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2025, pp. 170–180. (Citato alle pagine 1 e 10)
- [3] E. A. Napoli, F. Barbara, V. Gatteschi, and C. Schifanella, "Leveraging Large Language Models for Automatic Smart Contract Generation: the CO-STAR Case," in *Proceedings of the International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2024. (Citato alle pagine 1 e 2)
- [4] G. Iuliano, L. Allocca, M. Cicalese, and D. D. Nucci, "Automated vulnerability injection in solidity smart contracts: A mutation-based approach for benchmark development," 2025. [Online]. Available: <https://arxiv.org/abs/2504.15948> (Citato alle pagine 2, 7, 8 e 9)
- [5] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," <https://bitcoin.org/bitcoin.pdf>, 2008. (Citato a pagina 5)

-
- [6] G. Wood *et al.*, “Ethereum: A secure decentralised generalised transaction ledger,” *Ethereum project yellow paper*, vol. 151, no. 2014, pp. 1–32, 2014. (Citato alle pagine 5 e 6)
- [7] N. Szabo, “Formalizing and securing relationships on public networks,” *First monday*, 1997. (Citato a pagina 5)
- [8] R. Karanjai, E. Li, L. Xu, and W. Shi, “An empirical study of ai-based smart contract creation,” 2023. [Online]. Available: <https://arxiv.org/abs/2308.02955> (Citato alle pagine 6 e 8)
- [9] SWC Registry, “Smart contract weakness classification and test cases,” <https://swcregistry.io/>, 2024. (Citato alle pagine 6, 7 e 8)
- [10] OWASP Foundation, “Owasp smart contract top 10,” <https://owasp.org/www-project-smart-contract-top-10/>, 2023. (Citato a pagina 6)
- [11] Nomic Foundation, “Hardhat: Ethereum development environment for professionals,” <https://hardhat.org/docs/getting-started>. (Citato a pagina 8)
- [12] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. L. Traon, and M. Harman, “Chapter six - mutation testing advances: An analysis and survey,” ser. *Advances in Computers*, A. M. Memon, Ed. Elsevier, 2019, vol. 112, pp. 275–378. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0065245818300305> (Citato a pagina 9)
- [13] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Dang *et al.*, “Qwen2. 5-coder technical report,” *arXiv preprint arXiv:2409.12186*, 2024. (Citato a pagina 12)
- [14] M. Barboni, A. Morichetta, and A. Polini, “Sumo: A mutation testing approach and tool for the ethereum blockchain,” *Journal of Systems and Software*, vol. 193, p. 111445, 2022. (Citato alle pagine 21 e 22)
- [15] Z. Xiao, Y. Li, Q. Wang, and S. Chen, “Prompt to pwn: Automated exploit generation for smart contracts,” 2025. [Online]. Available: <https://arxiv.org/abs/2508.01371> (Citato a pagina 24)